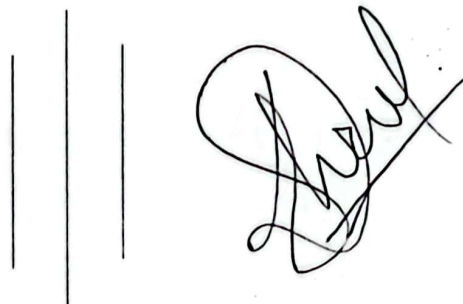




TRIBHUVAN UNIVERSITY
INSTITUTE OF SCIENCE AND TECHNOLOGY



HIMALAYA COLLEGE OF ENGINEERING
CHYASAL, LALITPUR



Lab Report No:- 4

Title:- Analysis of the Banker's Algorithm.

Submitted by:-

Name:- Sumajan Shrestha.

Roll No:- 38

Submitted To:- Dmish mali

Department Of CSIT

Checked by:-

Date of submission:- 2021/05/06

TITLE : Analysis of The Banker's Algorithm.

OBJECTIVE :

→ The objective of the lab is to implement the Banker's Algorithm in C++ to simulate deadlock avoidance in an operating system. The goal is to determine if the system is in a safe state by finding a safe sequence of processes.

THEORY :

→ The Banker's Algorithm is a resource allocation and deadlock avoidance algorithm developed by Edgar Dijkstra. It is primarily used in operating systems to allocate the resources to processes in such a way that a safe sequence of execution is always maintained. This ensures that the system remains in a state where all processes can complete their execution without causing a deadlock.

Key-Concepts :-

- • **Processes (P)** : The running programs or the tasks that require resources to execute.
- **Resources (R)** : The available resources in the system, which can be allocated to processes.
- **Allocated Matrix** : This matrix shows the no. of

resources of each type currently allocated to each process.

- **Max matrix**: This matrix shows the maximum number of resources of each type that each process may need.
- **Available matrix**: This vector indicates the no. of available resources of each type.
- **Need matrix**: This matrix is derived from the max and Allocation matrix and shows the remaining needed by each process to complete its execution.

Sample example :-

- Consider the system with 5 process P_0, P_1, P_2, P_3 & P_4 and let 3 resources be A, B, & C with instances 10, 5, 7 respectively. The following is the snapshot of the system is given:-

	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P_0	0	1	0	7	5	3	3	3	2
P_1	2	0	0	3	2	2			
P_2	3	0	2	9	0	2			
P_3	2	1	1	2	2	2			
P_4	0	0	2	4	3	3			

check if the system is in deadlock. If not find the safety sequence.

Soln:

Initially, finding Need matrix i.e. [max - Allocation]

	A	B	C	Available		
P ₀	7	4	3	A	B	C
P ₁	1	2	2	3	3	2
P ₂	6	0	0			
P ₃	0	1	1			
P ₄	4	3	1			

for process P₀,

Need P₀ > Available

Since, $[7, 4, 3] > [3, 3, 2]$.so, P₀ is not executed.

for process P₁,

Need P₁ < Available

$[1, 2, 2] < [3, 3, 2]$.so, P₁ is executed.

$$\begin{aligned}\therefore \text{Available} &= \text{Available} + \text{Allocation of P}_1 \\ &= [2, 0, 0] + [3, 3, 2] \\ &= [5, 3, 2]\end{aligned}$$

for process P₂,

Need P₂ > Available

Since, $[6, 0, 0] > [5, 3, 2]$.so, P₂ is not executed.

for process P₃,

Need P₃ < Available

Since, $[0, 1, 1] < [5, 3, 2]$.so, P₃ is executed

$$\begin{aligned}\therefore \text{Available} &= \text{Available} + \text{Allocation of P}_3 \\ &= [5, 3, 2] + [2, 1, 1] \\ &= [7, 4, 3]\end{aligned}$$

for P_4 , Need of $P_4 < \text{Available}$

$$[4, 3, 1] < [7, 4, 3]$$

$\therefore P_4$ is executed.

$$\begin{aligned}\therefore \text{Available} &= \text{Available} + \text{Allocation of } P_4 \\ &= [7, 4, 3] + [0, 0, 2] \\ &= [7, 4, 5]\end{aligned}$$

Now,

for P_0 , Need of $P_0 < \text{Available}$

$$[7, 4, 3] < [7, 4, 5]$$

$\therefore P_0$ is executed.

$$\begin{aligned}\text{Available} &= \text{Available} + \text{Allocation of } P_0 \\ &= [7, 4, 5] + [0, 1, 0] \\ &= [7, 5, 5]\end{aligned}$$

Again,

for P_2 , Need of $P_2 < \text{Available}$

$$[6, 0, 0] < [7, 5, 5]$$

$\therefore P_2$ is executed.

$$\begin{aligned}\therefore \text{Available} &= \text{Available} + \text{Allocation of } P_2 \\ &= [7, 5, 5] + [3, 0, 2] \\ &= [10, 5, 7]\end{aligned}$$

Now, all the process are executed, so there is no dead lock and the safety sequence is

$P_1 - P_3 - P_4 - P_0 - P_2$.

Safety check :

→ The Banker's Algorithm ensures safety by calculating a sequence in which all processes can complete without running out of resources. If such a sequence exists, the system is in a safe state.

Otherwise, it is unsafe, potentially leading to deadlock.

Steps of the Algorithm:

1. Initialization: The algorithm starts by calculating the Need matrix.
2. Check Process feasibility: For each process, check if its resource needs can be met by the available resources. If yes, assume that process completes, and add its allocated resources back to the available pool.
3. Safe sequence: Repeat this process for all processes. If all processes can complete, a safe sequence is found. Otherwise, the system is unsafe.

OBSERVATION :-

~~*/~~ // Source code:

// Banker's Algorithm

#include <iostream>

using namespace std;

int main()

{ int n, m, i, j, k;

n=5;

m=3;

int alloc[5][3] = { { 0, 1, 0 },

{ 2, 0, 0 },

{ 3, 0, 2 },

{ 2, 1, 1 },

{0, 0, 2} ;

```
int max [5] [3] = { { 7, 5, 3 },  
                    { 3, 2, 2 },  
                    { 9, 0, 2 },  
                    { 2, 2, 2 },  
                    { 4, 3, 3 } };
```

```
int avail [3] = { 3, 3, 2 };
```

```
int f[n]; ans[n], ind = 0;
```

```
for (k = 0; k < n; k++) { f[k] = 0; }
```

```
int need [n] [m];
```

```
for (i = 0; i < n; i++) {
```

```
    for (j = 0; j < m; j++)
```

```
        need [i] [j] = max [i] [j] - alloc [i] [j];
```

```
    }
```

```
    int y = 0;
```

```
    for (k = 0; k < 5; k++) {
```

```
        for (i = 0; i < n; i++) {
```

```
            if (f[i] == 0) {
```

```
                int flag = 0;
```

```
                for (j = 0; j < m; j++) {
```

```
                    if (need [i] [j] > avail [j]) {
```

```
                        flag = 1;
```

```
                        break;
```

```
                }
```

```
            }
```

```
            if (flag == 0) {
```

```
                ans [ind++] = i;
```

```
                for (y = 0; y < m; y++)
```



```
avail[y] += alloc[p][y];
```

```
f[p] = 1;
```

```
}
```

```
}
```

```
}
```

```
}
```

```
int flag = 1;
```

```
// To check if sequence is safe or not  
for (int p = 0; p < n; p++)
```

```
{ if (f[p] == 0)
```

```
{ flag = 0;
```

```
cout << "The given sequence is not safe";  
break;
```

```
}
```

```
}
```

```
if (flag == 1)
```

```
{ cout << "following is the safe sequence" << endl;
```

```
for (p = 0; p < n-1; p++)
```

```
cout << "p" << ans[p] << " → ";
```

```
cout << "p" << ans[n-1] << endl;
```

```
}
```

```
return (0);
```

```
}
```

Output :

following is the safe sequence

$p_1 \rightarrow p_3 \rightarrow p_4 \rightarrow p_0 \rightarrow p_2$


```
// Source - code :
```

```
// Bit map
```

```
#include <stdio.h>
```

```
#define size 8
```

```
int main()
```

```
{ int bitmap[size] = {0};
```

```
int choice, block;
```

```
do { printf("\n menu: \n");  
printf("1. Display Bitmap \n");  
printf("2. Allocate Block \n");  
printf("3. Free Block \n");  
printf("4. Exit \n");  
printf("\nEnter your choice:");  
scanf("%d", &choice);
```

```
switch(choice) {
```

```
case 1:
```

```
printf("Bitmap :");  
for (int i = 0; i < size; i++) {  
printf("%d", bitmap[i]);  
}  
printf("\n");  
break;
```

```
case 2:
```

```
printf("Enter block number to allocate (0-%d):",  
size - 1);  
scanf("%d", &block);  
if (block >= 0 && block < size) {  
if (bitmap[block] == 0) {  
bitmap[block] = 1;
```

```

printf("Block %d allocated.\n", block);
} else {
    printf("Block %d is already occupied.\n", block);
}
else {
    printf("Invalid block number.\n");
} break;

case 3:
    printf("Enter block number to free (0-%d):", size-1);
    scanf("%d", &block);
    if (block >= 0 && block < size) {
        if (bitmap[block] == 1) {
            bitmap[block] = 0;
            printf("Block %d freed.\n", block);
        } else {
            printf("Block %d is already free.\n", block);
        }
        printf("Invalid block number.\n");
    }
    break;

case 4:
    printf("Exiting ----\n");
    break;

default:
    printf("Invalid choice. Please try again.\n");
}

while (choice != 4);

return 0;
}

```

OUTPUT:

Menu:

1. Display Bitmap
2. Allocate Block
3. free Block
4. Exit

Enter your choice: 2

Enter block number to allocate (0-7): 3

Block 3 allocated.

Menu:

1. Display Bitmap
2. Allocate Block
3. free Block
4. Exit

Enter your choice: 1

Bitmap: 0 0 0 1 0 0 0 0

Menu:

1. Display Bitmap
2. Allocate Block
3. free Block
4. Exit

Enter your choice: 3

Enter your ~~at~~ block number to free (0-7): 3

Block 3 freed.

Menu:

1. Display Bitmap
2. Allocate Block
3. free Block
4. Exit

Enter your choice: 1

Bitmap : 00000000

DISCUSSION:

→ This lab exercise demonstrates the practical application of the Banker's Algorithm in ensuring safe resource allocation in operating systems. The algorithm is essential for preventing deadlock, a situation where processes are stuck waiting ~~for~~ ^{definitely} for resources held by each other. The implementation of the algorithm shows how the Need matrix is used to check the feasibility of resource allocation and how the system can transition between states safely. Understanding this algorithm is crucial for designing robust and efficient OS that can handle multiple processes and resource requests concurrently without entering a deadlock.

CONCLUSION:

→ The Banker's Algorithm was successfully implemented and tested. The program determined that the system is in a safe state by finding a sequence of processes ($P_1 \rightarrow P_3 \rightarrow P_4 \rightarrow P_0 \rightarrow P_2$) that can execute without causing a deadlock.